Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 29.01.2026.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test1_GrupaA_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Selection Sort*

Napisati ***Selection Sort*** algoritam za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomArrayGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- Kreirati klasu ***Selection*** i u okviru nje generisati metodu ***public static void Sort(long[] arr)***

Zadatak 2 – *Suma elemenata između prvog i poslednjeg pojavljivanja maksimalne vrednosti*

Implementirati metodu koja pronalazi **maksimalnu vrednost** u listi. Nakon toga, potrebno je izračunati i vratiti **sumu svih elemenata** koji se nalaze **između** prvog i poslednjeg pojavljivanja te maksimalne vrednosti (uključujući i te granične elemente).

Dodatni zahtevi za rešavanje zadatka:

- Ako se maksimalna vrednost pojavljuje samo jednom, suma je jednaka toj samoj vrednosti.
- Algoritam pisati u okviru klase ***LinkedList.***
- Ako lista ne postoji, vratiti 0.
- Algoritam mora imati vremensku kompleksnost **O(n)**.
- Zahtevan potpis metode: ***public int sumBetweenMaxBounds()***

Primer:

- Input: 4 → 2 → 3 → 4 ;
- Output: 13

- Input: 2 → 3 → 4 → 4 ;
- Output: 8

- Input: 2 → 3 → 1 → 4 ;
- Output: 0

Zadatak 3 – *Gravitaciona Drobilica (The Gravity Crusher)*

Zamisli da imaš vertikalnu cev u koju ubacuješ blokove različitih težina. Svaki blok ima svoju težinu (pozitivan ceo broj). Kada novi blok padne na onaj ispod njega, važe sledeća pravila:

1. **Drobljenje:** Ako je novi blok **strogo teži** od bloka na vrhu, on će ga smrskati (ukloniti iz steka) i pokušati da padne na sledeći blok ispod njega.
2. **Pucanje:** Ako je novi blok **iste težine** kao blok na vrhu, oba bloka pucaju i oba nestaju.
3. **Stabilnost:** Ako je novi blok **lakši** od bloka na vrhu, on jednostavno ostaje da leži na njemu.

Tvoj cilj: Implementirati metodu koja prima niz težina koje se redom ubacuju u cev i vraća finalno stanje blokova u cevi (od dna ka vrhu).

Dodatni zahtevi za rešavanje zadatka:

1. Vremenska složenost: **O(n)**.
2. Rezultat mora biti niz u originalnom redosledu (od pogo koji je ubačen i ostao na dnu, do poslednjeg na vrhu).

Primer:

Input 1 (nesortirani niz celih brojeva):	[3, 10, 2, 2, 8]
Output 1:	[10, 8]
Input 2 (nesortirani niz celih brojeva):	[7, 6, 10, 5, 4]
Output 2:	[10, 5, 4]